

Procesamiento de Imágenes en GPU: Gaussian Blur con OpenACC

CI-0117 Programación Paralela y Concurrente — Tarea 3

Gabriel Coto Fernández (C4E540)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
gabriel.cotofernandez@ucr.ac.cr

Jhon Campos Montenegro (C4D674)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
jhon.campos@ucr.ac.cr

Resumen—El objetivo del presente trabajo fue implementar un pipeline de procesamiento de imágenes el cual convierte una imagen a escala de grises y le aplica un filtro Gaussiano (Gaussian Blur), acelerado en GPU a través OpenACC y posteriormente, comparar los tiempos de ejecución en CPU y GPU obtenidos en el nodo Nukwa del clúster Kabré (CeNAT). Asimismo, se implementó una versión equivalente en CUDA explícito. En ambas variantes se obtuvo un speedup menor a 1, lo cual se explica principalmente por el costo de inicialización del contexto de GPU tal como se detalla en las secciones de resultados y discusión.

I. INTRODUCCIÓN

En lo que respecta al filtro Gaussian Blur, este consiste en suavizar una imagen reemplazando cada píxel por un promedio ponderado de sus vecinos, donde los pesos siguen una distribución Gaussiana en dos dimensiones. Es así que, para efectos del presente trabajo, dicho filtro se implementa sobre una imagen convertida previamente a escala de grises, contrastando su ejecución secuencial en CPU contra una versión acelerada en GPU mediante directivas OpenACC y adicionalmente contra una versión equivalente en CUDA explícito.

No se busca como tal determinar cuál de las dos tecnologías de aceleración (OpenACC o CUDA) es superior, sino observar en qué medida los tiempos medidos experimentalmente en el clúster Kabré corresponden con lo que se esperaría de una aceleración por GPU e identificar los factores que puedan explicar diferencias entre lo observado y lo esperado.

II. DESCRIPCIÓN DEL ALGORITMO

II-A. Kernel Gaussiano

En lo que respecta al kernel utilizado, se emplea uno de tamaño 3×3 normalizado (radio $R = 1$):

El filtro Gaussiano sustituye el valor de cada píxel por un promedio ponderado de sus vecinos, donde el peso asignado depende de una función Gaussiana bidimensional, de forma que el píxel central recibe el mayor peso y los píxeles más alejados reciben pesos progresivamente menores. En la práctica, dicha función continua se discretiza muestreando sus valores en el

centro de cada píxel del kernel y normalizando el resultado, de modo que la suma total de los pesos sea igual a uno [5].

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (1)$$

y de forma opcional un kernel 5×5 (radio $R = 2$), ambos con suma total igual a 1.

II-B. Convolución

Para cada píxel (y, x) de la imagen en escala de grises, el valor filtrado se obtiene mediante:

$$B(y, x) = \sum_{k_y=-R}^R \sum_{k_x=-R}^R K(k_y, k_x) I(y + k_y, x + k_x) \quad (2)$$

II-C. Manejo de bordes

Ahora bien, cuando el kernel se sale de los límites de la imagen se aplica la estrategia de *clamp*, es decir, se reutiliza el píxel válido más cercano:

```
int yy = std::min(std::max(y + ky, 0), height - 1);  
int xx = std::min(std::max(x + kx, 0), width - 1);
```

Cabe destacar que se mantienen dos arreglos separados (`gray[]` de entrada y `blur[]` de salida), de forma que se evita leer valores ya modificados durante el cómputo de la convolución.

III. IMPLEMENTACIÓN EN GPU (OPENACC)

OpenACC define tres niveles de paralelismo para la ejecución en el acelerador: *gang*, que corresponde a paralelismo de grano grueso; *worker*, de grano fino; y *vector*, para operaciones tipo SIMD o SIMT dentro de un worker [3, p. 9]. La especificación exige que el bucle más externo de un anidamiento se marque como *gang* y que el más interno se marque como *vector*, pudiendo aparecer opcionalmente un nivel *worker* entre ambos [4], lo cual justifica el uso de la directiva `collapse(2)` sobre el doble bucle de filas y columnas en la implementación descrita a continuación.

En cuanto a la aceleración en GPU se refiere, el filtro se paraleliza con OpenACC mediante una región de datos que transfiere la imagen y el kernel a la GPU una sola vez, seguida de un bucle paralelo doblemente anidado sobre filas y columnas:

```
#pragma acc data copyin(in[0:total], k[0:klen]) \
    copyout(out[0:total])
{
    #pragma acc parallel loop collapse(2) \
        present(in, out, k)
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            // convolucion (Ec. 2)
        }
    }
}
```

El compilador `nvc++` (NVIDIA HPC SDK 25.3) reporta, mediante la opción `-Minfo=accel`, que el bucle externo se paraleliza a nivel de gang (`collapse(2)`) y que el bucle interno del kernel se vectoriza, lo que confirma que el cómputo efectivamente se ejecuta sobre la GPU y no sobre el host.

IV. ENTORNO DE EJECUCIÓN EN KABRÉ

Las pruebas se ejecutaron en el clúster **Kabré** (CeNAT), sobre una imagen de prueba de 639×759 píxeles. Es importante aclarar un hallazgo relevante que surgió durante el proceso: los nodos de inicio de sesión de Kabré (`login-*`) no cuentan con GPU física ni con el driver de NVIDIA cargado, lo cual se verificó mediante `lsmod | grep nvidia`, comando que no devuelve ningún resultado en dichos nodos. Al ejecutar allí un binario compilado con OpenACC, el runtime no produce un error evidente, lo que en un primer momento llevó a interpretar tiempos que no correspondían a una ejecución real en GPU; no obstante, para CUDA sí se obtiene de forma explícita el error `CUDA driver version is insufficient for CUDA runtime version`.

Es por ello que resulta necesario solicitar una sesión interactiva en un nodo de cómputo con GPU real antes de compilar y ejecutar, para lo cual se emplea:

```
salloc --partition=nukwa-debug
```

Este comando asigna un nodo de la partición `nukwa` (en las pruebas reportadas, `nukwa-08`), donde `nvidia-smi` confirma la presencia de una GPU **NVIDIA L40S** real, con `compute capability` 8.9.

En consecuencia, los flags de arquitectura empleados en la compilación deben ser `-gpu=cc89` (OpenACC) y `-arch=sm_89` (CUDA), y no `cc70/sm_70` (Volta) como se había asumido inicialmente a partir de un ejemplo genérico que no correspondía al hardware real disponible.

La compilación de la versión OpenACC se realiza de la siguiente forma:

```
module load nvhpc/25.3
make acc
./gaussian_blur_acc images/iu.png output/
```

V. RESULTADOS: OPENACC

V-A. Tiempos y speedup

El cuadro I presenta los tiempos de ejecución obtenidos para el kernel 3×3 en el nodo Nukwa.

Tabla I
TIEMPOS DE EJECUCIÓN DEL GAUSSIAN BLUR (KERNEL 3×3) CON OPENACC EN KABRÉ, NODO NUKWA (GPU L40S REAL)

Métrica	Valor
Tiempo CPU (T_CPU)	16.4764 ms
Tiempo GPU (T_GPU)	185.581 ms
Speedup (T_CPU / T_GPU)	0.0888×

Al analizar estos resultados, el speedup obtenido resulta considerablemente menor a 1. Es importante aclarar que, a diferencia de una corrida previa realizada por error en un nodo de inicio de sesión sin GPU real, este resultado sí proviene de una ejecución verdadera sobre la GPU L40S de Nukwa. La causa más probable del alto T_GPU observado es el costo de **inicialización del contexto de GPU** (creación del contexto CUDA subyacente y compilación JIT del código para el dispositivo real), el cual ocurre una única vez por proceso en el primer lanzamiento de kernel.

Esta explicación es consistente con el comportamiento documentado del propio runtime de NVIDIA: el driver de Linux apaga las GPU inactivas cuando no existe ningún contexto CUDA abierto, y reactivarlas puede tardar hasta un segundo, lo cual se traduce en retrasos perceptibles al inicio de la ejecución del programa; dicho tiempo de arranque es reportado explícitamente como “tiempo de inicialización” cuando se habilita la variable de entorno `NVCOMPILER_ACC_TIME` [2, p. 38]. Asimismo, la compilación *just-in-time* (JIT) del código PTX a código nativo del dispositivo introduce una sobrecarga adicional de arranque en la aplicación, la cual el compilador mitiga en ejecuciones posteriores mediante un caché de binarios compilados [2, p. 11].

Dado que el programa realiza una sola llamada a `gaussian_blur_gpu` antes de finalizar, dicho costo de arranque queda íntegramente contenido dentro de la medición de T_GPU.

V-B. Capturas



Figura 1. Imagen original.



Figura 2. Imagen en escala de grises.



Figura 3. Imagen tras aplicar Gaussian Blur (3×3) en GPU (OpenACC), nodo Nukwa.

VI. IMPLEMENTACIÓN ADICIONAL: CUDA

Como parte opcional del trabajo, se implementó el mismo filtro Gaussiano utilizando CUDA explícito en lugar de directivas OpenACC. A diferencia de OpenACC, en donde es el compilador quien decide la estrategia de paralelización, en CUDA dicho control se realiza de forma manual: la transferencia de datos (`cudaMemcpy`), la configuración de bloques e hilos, y el lanzamiento del kernel.

VI-A. Archivos agregados

En cuanto a la integración con el proyecto existente se refiere, la implementación se agregó sin modificar el código ni los targets de compilación ya existentes (`all`, `acc`):

- `include/cuda_blur.hpp`: declaración de `gaussian_blur_cuda`, con la misma firma que las versiones CPU/OpenACC.
- `src/cuda_blur.cu`: kernel `__global__` y manejo de memoria del device.
- `src/main_cuda.cpp`: pipeline equivalente a `main.cpp`, invocando la versión CUDA.
- Nuevo target `cuda` en el `Makefile`, que compila con `nvcc`.

VI-B. Kernel CUDA

Se define un kernel `__global__` en el que cada hilo calcula un único píxel de salida, replicando la misma convolución

y el mismo manejo de bordes (`clamp`) que las versiones CPU y OpenACC:

```
__global__ void blur_kernel(const unsigned char* in,
                           unsigned char* out,
                           const float* k, int
                               width,
                               int height, int radius,
                               int ksize) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) return;
    // convolucion con clamp de bordes (Ec. 2)
}
```

EL uso de la memoria se maneja de forma explícita mediante `cudaMalloc` y `cudaMemcpy` (sin memoria unificada), realizando una única transferencia de entrada y una única transferencia de salida por ejecución, esto con el propósito de minimizar el overhead de comunicación entre host y device. Asimismo, cada llamada a la API de CUDA se envuelve en un macro de verificación de errores (`CUDA_CHECK`) que imprime el mensaje real del driver en caso de que alguna llamada falle, en lugar de continuar en silencio.

VI-C. Proceso de compilación y depuración en Kabré

La compilación de esta parte requirió resolver varios problemas encontrados durante el proceso, los cuales se documentan a continuación por transparencia:

1. **Arquitectura de GPU incorrecta.** La compilación inicial empleó `-arch=sm_70` (Volta), heredado por analogía del flag `-gpu=cc70` utilizado en OpenACC. Al confirmar mediante `nvidia-smi` que el nodo Nukwa real (`nukwa-08`) posee una GPU **L40S** (Ada Lovelace, *compute capability* 8.9), se corrigió el flag a `-arch=sm_89`.
2. **Ejecución en nodo sin GPU.** Al ejecutar el binario inicialmente en el nodo de inicio de sesión (`login-0`), se obtuvo el error `CUDA driver version is insufficient for CUDA runtime version` en cada llamada a la API de CUDA. Dicho error solo se hizo visible tras agregar la macro `CUDA_CHECK`; sin ella, el programa continuaba de forma silenciosa con buffers de memoria en cero, lo que producía una imagen de salida completamente negra. La causa raíz es que los nodos de login no cuentan con GPU física, tal como se confirmó con `lsmod | grep nvidia` (sin resultados). La solución consistió en solicitar una sesión interactiva en un nodo de cómputo real mediante `salloc --partition=nukwa-debug`.

La compilación y ejecución final dentro de la sesión interactiva en `nukwa-08`, se realiza de la siguiente forma:

```
salloc --partition=nukwa-debug
module load cuda/12.4.0
make cuda
./gaussian_blur_cuda images/iu.png output/
```

Tabla II
TIEMPOS DE EJECUCIÓN DEL GAUSSIAN BLUR CON CUDA EXPLÍCITO
(KERNEL 3×3) EN KABRÉ, NODO NUKWA (GPU L40S REAL)

Métrica	Valor
Tiempo CPU (T_CPU)	22.6722 ms
Tiempo GPU (T_GPU)	189.563 ms
Speedup (T_CPU / T_GPU)	$0.1196\times$

A diferencia de una medición anterior, también inválida, obtenida por error en el nodo de login sin GPU real (en la cual se había registrado un speedup de $6,03\times$), el resultado real obtenido en la GPU L40S de Nukwa muestra un T_GPU similar en magnitud al de OpenACC (cuadro I: 185.58 ms frente a 189.56 ms en este caso), a pesar de tratarse de dos tecnologías distintas con manejo de memoria diferente. Esta similitud refuerza la hipótesis de que el costo dominante en ambos casos no corresponde al algoritmo de convolución en sí ni a la estrategia de paralelización empleada, sino al costo fijo de **inicialización del contexto de GPU** en el primer lanzamiento de kernel del proceso, el cual resulta comparable entre OpenACC y CUDA al tratarse ambos de procesos independientes de una sola invocación.



Figura 4. Imagen en escala de grises (corrida CUDA).



Figura 5. Imagen tras aplicar Gaussian Blur (3×3) con CUDA explícito, nodo Nukwa.

VII. CONCLUSIONES

Con respecto al objetivo del trabajo, este consistió en implementar el pipeline de lectura de imagen, conversión a escala de grises y Gaussian Blur, con dos variantes de aceleración en GPU (OpenACC y CUDA explícito), ambas validadas mediante ejecución real en el nodo Nukwa de Kabré (GPU NVIDIA L40S). Dicho objetivo se alcanzó para ambas variantes, aunque en las dos se obtuvo un speedup menor a 1 ($0.089\times$ en OpenACC, $0.120\times$ en CUDA) para la imagen de prueba utilizada (639×759 píxeles, kernel 3×3).

El hallazgo más relevante del proceso de medición es que el T_GPU reportado en ambas variantes parece estar dominado por el costo de inicialización del contexto de GPU en el primer lanzamiento de kernel, más que por el trabajo de convolución en sí, el cual es computacionalmente liviano para una imagen de este tamaño. Esto se evidencia en la similitud de T_GPU entre OpenACC y CUDA a pesar de sus diferencias de implementación, y en que el propio timer interno de la versión CUDA (medido después de las llamadas iniciales a `cudaMalloc`) resulta del orden de 500 veces menor que el T_GPU reportado externamente.

REFERENCIAS

- [1] M. Harris, “CUDA Pro Tip: Understand Fat Binaries and JIT Caching,” *NVIDIA Developer Blog*, 2022. Recuperado de: <https://developer.nvidia.com/blog/cuda-pro-tip-understand-fat-binaries-jit-caching/>
- [2] NVIDIA Corporation, *OpenACC Getting Started Guide*, Release 26.1. NVIDIA, 2026. Recuperado de: <https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/>
- [3] OpenACC-Standard.org, *OpenACC Application Programming Interface*, Version 2.0. 2013. Recuperado de: https://www.openacc.org/sites/default/files/inline-files/OpenACC_2_0_specification.pdf
- [4] OpenACC-Standard.org, *OpenACC Programming and Best Practices Guide*. 2021. [En línea]. Disponible: <https://openacc-best-practices-guide.readthedocs.io/>
- [5] R. Fisher, S. Perkins, A. Walker y E. Wolfart, “Spatial Filters — Gaussian Smoothing,” *HIPR2*, University of Edinburgh. Recuperado de: <https://homepages.inf.ed.ac.uk/rbf/HIPR2/gsmooth.htm>